
Algorithm 7: Smart-Greedy Rollout Probe with Tabu List

Input:

- L_{start} . The starting label $(v, t, C, \text{pred}, \text{start_time})$
- E_s . Set of scheduled edges $(v, u, \text{dep}, \text{arr})$
- E_t . Set of transfer edges (v, w, Δ)
- $\text{country}(v)$. Country associated with node v

Output:

- L_{final} . The final label reached when the 24-hour clock expires

```

1:  $L \leftarrow L_{\text{start}}$ 
2:  $\text{RolloutVisitedNodes} \leftarrow \{L.\text{node}\}$  // Tabu list to prevent all local loops
3: // Repeat until no more valid moves within 24 hours
4: repeat
5:    $\text{BestNodeArrivals} \leftarrow \text{Empty Map}$  // Maps a node 'u' to the earliest arrival time 'tnew' and
   departure 'dep'
6:   // 1. Collect ONLY the earliest arrival for each neighboring node
7:   for all  $(L.\text{node}, u, \text{dep}, \text{arr}) \in E_s$  do
8:     if  $\text{dep} \geq L.t$  &  $\text{arr} - L.\text{start\_time} \leq 1440$  then
9:       if  $u \notin \text{BestNodeArrivals}$  or  $\text{arr} < \text{BestNodeArrivals}[u].\text{arr}$  then
10:         $\text{BestNodeArrivals}[u] \leftarrow (\text{dep}, \text{arr}, \text{country}(u))$ 
11:   for all  $(L.\text{node}, w, \Delta) \in E_t$  do
12:      $t_{\text{new}} \leftarrow L.t + \Delta$ 
13:     if  $t_{\text{new}} - L.\text{start\_time} \leq 1440$  then
14:       if  $w \notin \text{BestNodeArrivals}$  or  $t_{\text{new}} < \text{BestNodeArrivals}[w].\text{arr}$  then
15:         // For transfers, dep and arr are both tnew
16:          $\text{BestNodeArrivals}[w] \leftarrow (t_{\text{new}}, t_{\text{new}}, \text{country}(w))$ 
17:   if  $\text{BestNodeArrivals}$  is empty then
18:     break // No valid forward moves left
19:   // 2. Assign heavily biased weights to the deduplicated list
20:    $\text{ValidSteps} \leftarrow \emptyset$ 
21:    $\text{curr\_country} \leftarrow \text{country}(L.\text{node})$ 
22:   for all  $u \in \text{BestNodeArrivals}$  do
23:      $(\text{dep}, \text{arr}, c_{\text{new}}) \leftarrow \text{BestNodeArrivals}[u]$ 
24:      $\Delta t \leftarrow \max(1, \text{arr} - L.t)$  // Total time consumed by this step
25:     if  $c_{\text{new}} \notin L.C$  then
26:       // UNVISITED COUNTRY: Absolute highest priority.
27:        $\text{base\_weight} \leftarrow 100,000$ 
28:     else if  $c_{\text{new}} \neq \text{curr\_country}$  then
29:       // TRANSIT VIA OTHER COUNTRY: Crucial for escaping geographical traps.
30:        $\text{base\_weight} \leftarrow 1,000$ 
31:     else
32:       // DOMESTIC TRANSIT: Just used to get to hubs.
33:        $\text{base\_weight} \leftarrow 1$ 
34:      $\text{weight} \leftarrow \text{base\_weight} / \Delta t$  // Linear time penalty
35:     // THE TABU PENALTY: Prevent wandering in local loops
36:     if  $u \in \text{RolloutVisitedNodes}$  then
37:        $\text{weight} \leftarrow \text{weight} \times 0.0001$ 
38:      $\text{ValidSteps} \leftarrow \text{ValidSteps} \cup \{(u, \text{arr}, c_{\text{new}}, \text{weight})\}$ 

```

```

39: | // 3. Roll the weighted dice
40: |  $s_{\text{chosen}} \leftarrow \text{WeightedRandomChoice}(\text{ValidSteps})$ 
41: | // 4. Update State
42: |  $C' \leftarrow L.C \cup \{s_{\text{chosen}}.c_{\text{new}}\}$ 
43: |  $\text{RolloutVisitedNodes} \leftarrow \text{RolloutVisitedNodes} \cup \{s_{\text{chosen}}.u\}$ 
44: |  $L \leftarrow (s_{\text{chosen}}.u, s_{\text{chosen}}.\text{arr}, C', \text{pred} = L, \text{start\_time} = L.\text{start\_time})$ 
45: | until false
46: | return  $L$ 

```

From previous algorithms execution, we observed a quick raise to 11-12 countries, but then the algorithm would quickly get stuck. This is most likely because too solutions are very similar, can't get discarded because of the soft relaxed conditions and UB pruning, and we keep expanding many labels that don't result useful but don't have a way to discard.

This concept wants to help us deal with it the following way: every 1000 iterations, we will pause the queue, pick the current label and run a "randomized" greedy rollout to see if we can find a better solution. Since it does 0 heavy math, it reaches the 24-hour mark in a matter of milliseconds, and if we get lucky and find a better solution, we can update the global record and prune the queue massively. Likely this won't be enough to consider the algorithm as finalized, because it is unlikely that making greedy jumps we find a better solution. However, it will certainly not decrease current performance as these rollouts take almost 0 time, and in the best case we can save a huge amount of time.

To avoid making a random rollout, which would yield in most cases no improvement in countries or a very low one (it could easily jump to many stations within the same country, specially if it's in one with many stations), we apply some rules to try and get a smarter approach. These are:

- Instead of looking at all departing edges from that node, look at the faster edge to every neighboring node. This kills volume.
- We give a heavy weight to transports that lead us to visit a new, unvisited country, so the rollout prioritizes these new routes
- We forbid the rollout to take an edge back to any of the nodes it has visited in the current rollout.